NYC 311 Workforce Optimization

Technical problem definition, mathematical formulation, solution method, and implementation notes for two hourly staffing dashboards

Dashboard	Application entry point	Local URL
Hourly Staffing Optimizer	src/workforce_app.py	http://127.0.0.1:8051/
Supervisor Workforce Control	src/workforce_control_app.py	http://127.0.0.1:8052/

Executive summary

Both applications turn recent, borough-filterable NYC 311 service-request creation aggregates into a 24-hour call-demand scenario, then solve a budget-constrained mixed-integer staffing problem. The optimizer assigns whole shifts, allocates integer calls to staffed agents by hour, respects hourly handling capacity and per-shift workload limits, and records calls that cannot be covered. Its objective is lexicographic: first minimize same-hour unserved demand; next minimize labor cost without increasing the best service shortfall found in the first stage.

Critical interpretation: NYC Open Data contains service requests, not call-center ACD arrivals, employee schedules, agent handling times, or payroll. Calls-per-request, staff availability, wage, and capacity controls are scenario inputs. Outputs are planning aids, not approved rosters or measured labor savings.

1. Problem and decision context

The planning problem is to match a forecast of hourly arrivals to a set of feasible shifts. A plan should cover as much demand as possible within the daily budget and safe workload assumptions, while identifying rather than concealing any service shortfall. The dashboards work at one-hour resolution for a single planning day. They do not simulate a live queue.

Element	Model representation	Interpretation
Demand	D[t], t = 0,...,23	Estimated calls arriving in each hour
Assignment	y[i,s] in {0,1}	Agent i is assigned to candidate shift s
Call allocation	x[i,t] integer >= 0	Calls assigned to agent i in hour t
Shortfall	u[t] integer >= 0	Calls not assigned in hour t
Cost	c[i,s] * y[i,s]	Cost of the complete assigned shift

For this model, unserved means not covered in the same hour. It is not automatically a lost call or a carry-over backlog. A queueing formulation would need state balance, abandonment, service levels, and arrivals during the interval.

2. Data flow and demand forecasting

Both apps use **nyc311_data.load_recent_hourly_requests**. At startup, the loader requests up to 120 recent days from the NYC Open Data Socrata resource **erm2-nwe9**, grouping at source by request creation date, hour, and borough. It normalizes missing borough values, sorts the aggregates, and reports the newest source date. Each dashboard also has a refresh action that reloads the grouped series; a refresh error is surfaced rather than silently being described as current.

```
Source aggregate: R[d,h,b] = count of requests created on date d, hour h, borough b.
```

The selected borough is aggregated across borough labels for ALL, or filtered to one borough. A complete daily-by-hour matrix is built for forecasting, reindexed to 24 hours per date; missing date/hour combinations are filled with zero. Therefore, source completeness and the request timestamp definition matter to both forecast variants.

2.1 Hourly Staffing Optimizer: same-weekday mean baseline

The baseline in **src/workforce_optimizer.py::forecast_hourly_demand** uses the mean of observations at each hour from the preceding 56 calendar days having the target date's weekday. At least 28 days of history are required, and a target date must be after the latest observation. The mean request count is rounded to a non-negative integer, then converted to calls using the editable factor (default 0.05 calls per request).

```
Rhat[t,h] = mean{ R[d,h] : t-56 days <= d < t, weekday(d)=weekday(t) }
D[t,h] = max(0, round( Rhat[t,h] * q )), q = calls per request
```

Rolling-origin validation uses past dates, never future observations for a validation prediction. The validation window is the later of the final 28 observed dates or the first date with 56 days of history. For each validation date, the predictor uses only its preceding 56 days with the same weekday. The dashboard reports MAE and RMSE in scenario calls per hour, with the number of hourly observations.

2.2 Supervisor Workforce Control: weekday/hour upper-quantile baseline

The supervisor app calls **src/workforce_control.py::forecast_control_demand**. For each hour, it selects the preceding 56 days with the target weekday and uses the 75th percentile (configurable internally within 0.50-0.99). At least four matching weekdays must exist. This is a deliberately more conservative baseline than a mean, not a learned time-series model.

```
Qhat[t,h] = quantile_0.75{ R[d,h] : t-56 days <= d < t, weekday(d)=weekday(t) }
Rproxy[t,h] = max(0, round(Qhat[t,h]))
Dbase[t,h] = max(0, round(Rproxy[t,h] * q))
Dbuffer[t,h] = round(Dbase[t,h] * beta), 1.00 <= beta <= 1.30
```

This dashboard defaults to q=0.05 and beta=1.12. It validates against rolling-origin dates from the latest 28 calendar days, skipping dates without four prior matching weekdays. MAE and RMSE are calculated in calls per hour. The buffer is applied after the point forecast and before optimization; it is a scenario reserve, not a statistical prediction interval.

The two models should not be conflated: the Hourly Optimizer uses a same-weekday mean; Workforce Control uses a same-weekday 75th percentile plus an explicit demand buffer. Their recommended staffing and shortfalls can differ even with identical cost/capacity controls.

3. Mathematical formulation

Sets and inputs

Symbol	Definition
T	Hourly periods {0,...,23}.
I	Available call takers; roster rows when supplied, otherwise interchangeable agents.
S[i]	Feasible shift start times for agent i; shifts are wholly within the day.
D[t]	Forecast or buffered call demand in hour t.
L	Shift length in hours.
a[i,s,t]	1 when shift s places agent i on duty in hour t; otherwise 0.
H	Hourly call capacity at the configured handling time and occupancy.
c[i,s]	Hourly wage for agent i multiplied by shift length.
Cmin[i], Cmax[i]	Optional minimum and maximum total assigned calls for agent i's completed shift.
B	Daily labor budget.

Decision variables

```
y[i,s] in {0,1} 1 if agent i is assigned to shift s
x[i,t] in Z+, calls assigned to agent i during hour t
u[t] in Z+, forecast calls not covered during hour t
```

Lexicographic objective

The implementation solves two MILPs sequentially rather than relying on a hand-tuned big-M coefficient. Stage 1 finds the smallest total shortfall. Stage 2 minimizes full-shift cost subject to shortfall no worse than the Stage 1 result. This prioritizes modeled service coverage over cost while respecting the budget in both stages.

```
Stage 1: minimize U = sum[t in T] u[t]
Stage 2: minimize C = sum[i in I] sum[s in S[i]] c[i,s] y[i,s]
subject to all constraints and sum[t in T] u[t] <= U*
```

Constraints

```
(1) Demand balance, each hour:
sum[i in I] x[i,t] + u[t] = D[t] for all t
(2) On-duty hourly capacity:
x[i,t] <= H[i,t] * sum[s in S[i]] a[i,s,t] y[i,s] for all i,t
(3) Per-shift maximum workload:
sum[t in T] x[i,t] <= Cmax[i] * sum[s in S[i]] y[i,s] for all i
(4) Optional per-shift minimum workload:
sum[t in T] x[i,t] >= Cmin[i] * sum[s in S[i]] y[i,s] for all i
(5) At most one shift per agent:
sum[s in S[i]] y[i,s] <= 1 for all i
(6) Daily budget:
sum[i,s] c[i,s] y[i,s] <= B
```

4. Capacity, cost, and solver implementation

Hourly capacity and midpoint break

In the Hourly Optimizer, hourly capacity is computed from average handle time and a configured occupancy ceiling: H = (60 / AHT_minutes) * occupancy_fraction. The default inputs are eight minutes and 85% occupancy, giving 6.375 calls per agent-hour before the model's integer allocations. In Workforce Control, the user directly sets maximum calls per agent-hour (default 8); the equivalent AHT/occupancy derivation is not applied there.

```
H[i,t] = H * b[i,s,t]
b[i,s,t] = 0.5 when t = s + floor(L/2), otherwise 1.0, for an assigned shift
x[i,t] <= H[i,t] * active[i,t]
```

The selected shift has one modeled midpoint hour at half capacity, intended as a simple proxy for a 30-minute break. It is not a detailed sub-hour break schedule and does not verify labor rules or rest compliance. If a shift starts at 16:00 and lasts eight hours, its end is 24:00; no cross-midnight shift is permitted.

Shift cost and workload

```
c[i,s] = hourly_wage[i] * L
C = sum of complete assigned-shift costs, not hourly headcount cost
Cmin[i], Cmax[i] constrain total calls over the full shift, not each hour
```

When no roster is uploaded, the Hourly Optimizer creates the configured number of interchangeable agents with shared eligible starts, wage, and workload limits. Its optional CSV roster has agent_id, eligible_shifts, hourly_wage, and max_calls_per_shift; min_calls_per_shift is optional and defaults to zero. Agent shift eligibility and costs are then individual.

Numerical solution procedure

Step	Implementation
1. Construct variables	Binary y for each feasible agent/start; integer x for each agent/hour and u for each hour.
2. Build sparse constraints	SciPy sparse COO/CSR matrix and LinearConstraint encode demand, capacity, workloads, one shift, and budget.
3. Minimize shortfall	scipy.optimize.milp with integer variables and unit objective coefficient on each u[t].
4. Minimize cost	Second milp minimizes shift costs and constrains total u to be no more than the first-stage result.
5. Report	Return hourly coverage/capacity, selected shifts and workloads, cost, budget remaining, shortfall, and solver status.

The solver is SciPy's HiGHS MILP backend. The implementation uses a 30-second time limit and 1% relative MIP gap. If Stage 2 produces no solution vector, Stage 1's solution is retained. Consequently, always inspect the returned solver status; a time-limited incumbent is not proof of a globally optimal schedule.

5. Dashboard-specific implementation

5.1 Hourly Staffing Optimizer (port 8051)

Entry point: **src/workforce_app.py**. Forecasting, roster parsing, and optimization live in **src/workforce_optimizer.py**. The app loads 120 recent days at startup and can refresh live grouped records. It accepts borough, target date, calls-per-request factor, available agents, eligible shift starts, shift length, daily budget, wage, average handle time, occupancy limit, per-shift max/min workload, and optional roster CSV.

Control / output	Default or semantics
Request-to-call factor	0.05; scenario conversion, not a measured NYC311 ratio.
Generic agent pool	35 agents; shift starts 00, 04, 08, 12, 16; 8-hour shifts.
Budget / wage	$5,500 daily budget; $25 per hour, unless roster rates are supplied.
Capacity	8-minute handle time; 85% occupancy; one midpoint break at 50% capacity.
Per-shift calls	Maximum 48; optional minimum defaults to 0.
Results	Forecast validation, demand/served/shortfall/staffing chart, selected shifts, hourly table, spend, remaining budget, solver status.

5.2 Supervisor Workforce Control (port 8052)

Entry point: **src/workforce_control_app.py**; forecast helper: **src/workforce_control.py**; MILP is shared with the Hourly Optimizer. Its forecast uses a 56-day same-weekday 75th percentile and then applies an editable 100%-130% demand buffer. Controls include target date, borough, budget, direct hourly cap, buffer, max/min shift calls, calls-per-request factor, available agents, and wage. The shift length is eight hours, starts are fixed at 00/04/08/12/16, and midpoint capacity is halved.

Control / output	Default or semantics
Budget / wage	$15,500 daily; $31.50 per hour.
Demand assumptions	8 calls/agent-hour; 112% buffer; 0.05 calls/request.
Generic agent pool	35 agents; five fixed eight-hour shift starts.
Per-shift calls	Maximum 52; minimum 0 by default.
Results	Request proxy, shifts, spend and remaining budget, uncovered buffered demand, coverage percentage, safe capacity chart, shift summary, validation, JSON export.
Release treatment	Plan is marked ready for review only when solver status is optimal and no buffered demand is uncovered; export remains a review artifact.

The control app distinguishes baseline forecast calls from buffered calls sent to the optimizer. Its charts present baseline demand, buffered demand, staffed handling capacity, and uncovered buffered calls. The displayed shift table groups assignments by shift start; JSON export retains scenario configuration, hourly results, assignments, validation metrics, and source date.

6. Reading results and operational use

Measure	Definition / correct interpretation
Forecast request proxy	Hourly service requests expected by the chosen statistical baseline; not ACD calls.
Forecast calls	Rounded request proxy multiplied by calls-per-request scenario factor.
Buffered calls	Workforce Control forecast calls multiplied by the selected reserve factor and rounded; only this enters that app's solver.
Calls served / covered	Integer calls assigned to agents in each hour, constrained by shift and capacity.
Unserved / uncovered	Same-hour unmet modeled demand; no carry-forward queue is included.
Staffing spend	Sum of full costs for assigned shifts. It is not a payroll forecast unless wages and rules are calibrated.
Budget remaining	Configured daily budget minus modeled shift cost.
Coverage percent	Workforce Control: (buffered total - uncovered) / buffered total; when demand is zero it displays 100%.
MAE / RMSE	Rolling-origin hourly forecast errors in scenario calls/hour; retrospective diagnostics, not a guarantee.

Recommended scenario workflow

- Refresh the request source and verify the latest source date and selected borough.
- Replace the request-to-call conversion with a factor calibrated from matched ACD calls and service requests; do not infer the factor from the dashboard itself.
- Provide authorized roster availability, shift eligibility, wage, and per-agent workload limits where applicable.
- Set handling capacity from measured handle time and an approved occupancy ceiling; separately validate break placement and staffing policies.
- Choose budget and demand reserve, calculate the plan, and review hourly shortfalls and solver status before considering the shifts.
- Treat the output as a candidate planning artifact. A supervisor must validate the schedule, service obligations, employee availability, and labor rules before release.

7. Assumptions, exclusions, and model risks

Included simplification	What it excludes / consequence
24 independent hourly demand balances	No queue/backlog state, wait time, abandonment, callback, or call transfer dynamics.
One-day candidate shifts	No overnight/cross-midnight shift, multi-day rest, overtime, or locked in-progress shifts.
Single pooled service capacity	No skills, language groups, call types, team routing, or priority classes.
One reduced-capacity midpoint hour	No actual break windows, paid/unpaid break rules, or sub-hour schedule.
Fixed hourly capacity and shift caps	No stochastic handle-time distribution, occupancy dynamics, shrinkage profile, or agent-specific speed unless represented by roster caps.
Deterministic forecast and budget	No robust/stochastic optimization, forecast confidence intervals, or explicit uncertainty penalties.
Request proxy for call demand	Request arrivals can differ materially from phone volume; conversion assumptions can bias staffing.
Hourly wage times shift hours	No benefits, differentials, overtime premiums, payroll overhead, or schedule-specific compensation rules.

The optional Cmin applies only to the total calls assigned to an agent over a selected shift. It is zero by default because low demand should not force artificial productivity quotas. A positive minimum may make the problem infeasible or distort allocations when feasible work is scarce. Cmax is also per completed shift; the separate hourly capacity constraint remains necessary to prevent excessive hourly load.

8. Run, inspect, and maintain

Dashboard	Launch from repository root
Hourly Staffing Optimizer	.venv/bin/python src/workforce_app.py
Supervisor Workforce Control	.venv/bin/python src/workforce_control_app.py

Both apps default to loopback access. Open ports 8051 and 8052 respectively. Relevant implementation files are **src/nyc311_data.py** (source aggregation), **src/workforce_optimizer.py** (baseline forecast, roster parsing, MILP), **src/workforce_control.py** (percentile forecast and backtest), **src/workforce_app.py**, and **src/workforce_control_app.py**. Automated MILP and forecast regression tests are in **tests/test_workforce_optimizer.py** and **tests/test_workforce_control.py**.

To regenerate this standalone PDF after changing the model or its documented controls, run **.venv/bin/python scripts/generate_workforce_technical_pdf.py**. Confirm the generated file opens, its page count is reasonable, and the formulas and assumptions remain consistent with the code.

Prepared 24 October 2026. Values in this document describe application defaults and source-code behavior at preparation time; live NYC Open Data values continue to change.